53

BÖLÜM: EŞZAMANLILIK HATALARI, TEŞHIS & DÜŞÜK SEVIYELI PRIMITIFLER | MODULE: CONCURRENCY BUGS, ANTI-PATTERNS & LOW-LEVEL PRIMITIVES

# Production Thread Dump Analizi: jstack, jcmd, Thread Durumları & CPU Tavanlarını Teşhis

Production Thread Dump Analysis: jstack, jcmd & Diagnosing CPU Spikes

## ⚡ Bellek & Eşzamanlılık Akış Şeması

## ⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                PRODUCTION THREAD DUMP ADLİ TIP REHBERİ       │
└──────────────────────────────────────────────────────────────┘

  CANLI SUNUCUDA CPU %100 VEYA KİLİTLENME VARSA:

  1. PID TESPİTİ:    top -H -p <PID> ──► En çok CPU yiyen Thread ID'yi bul (ör. 12948)
  2. HEX DÖNÜŞÜMÜ:   printf '%x\n' 12948 ──► "0x3294" (nid formatı)
  3. DUMP ALMA:      jcmd <PID> Thread.print > dump.tdump (veya jstack -l <PID>)
  4. NID ARAMA:      grep -A 20 "0x3294" dump.tdump ──► Suçlu metod ve satır karşında!

  DUMP'TA NE ARANIR?
  - "waiting to lock <0x00...>" ──► Kilidi kim tutuyor?
  - "locked <0x00...>"          ──► Kilidi tutan thread ne yapıyor?
  - "Found 1 deadlock."         ──► HotSpot deadlock'u otomatik listeler!
```

## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Thread dump, bir binadaki yangın anında bütün çalışanların dondurulup röntgenlerinin çekilmesidir. Kim elinde yangın tüpüyle bekliyor, kim kapıyı arkadan kilitlemiş, kim koridorda boşuna koşup terliyor tek bir fotoğrafta görünür. `top` komutuyla en çok terleyen personeli bulursun, thread dump ile de o personelin tam olarak hangi odada hangi kâğıdı imzalamaya çalıştığını saniyesi saniyesine yakalarsın!

### 🇬🇧 ENGLISH (ELI5)

A Thread Dump is an instantaneous radiographic snapshot of every single worker in your distributed factory. It captures who is sleeping, who is fighting over a wrench, and who is frantically running on a treadmill. Using `top -H` you pinpoint the exact OS thread ID consuming 100% CPU, convert it to hexadecimal (`nid`), and search the thread dump to reveal the exact Java file and line number executing the rogue loop.

## 🔍 MAANG Seviyesi Teknik Derinlik

### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot Thread Dump Anatomisi ve Teşhis Teknikleri: 1) Üretim Komutları: Modern JDK'larda `jstack` yerine `jcmd \<PID> Thread.print -l` tercih edilir. Sanal thread'leri dökmek için `jcmd \<PID> Thread.dump_to_file -format=json dump.json` kullanılır. 2) Dump Başlığı Çözümleme: `"http-nio-8080-exec-12" #45 daemon prio=5 os_prio=0 tid=0x7f nid=0x3294 waiting on condition [0x7000]`. Burada `nid` (Native Thread ID) işletim sistemi thread ID'sinin hex karşılığıdır. 3) Kilit Zincirleri: `java.lang.Thread.State: BLOCKED (on object monitor)` görüyorsanız `waiting to lock \<0x00007f...>` adresini dump içinde aratıp o adresi `locked \<0x00007f...>` olarak elinde tutan thread bulunur. 4) Deadlock Tespiti: HotSpot JVM dökümün en altında `Found one Java-level deadlock` başlığıyla kilit döngüsünü otomatik raporlar.

## 🔍 MAANG-Level Technical Deep-Dive

### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Thread Dump Anatomy & Diagnostic Protocol: 1) Modern Extraction Tooling: Prefer `jcmd \<PID> Thread.print -l` over legacy `jstack`. To analyze virtual threads, invoke `jcmd \<PID> Thread.dump_to_file -format=json threads.json`. 2) Header Dissection: In `"pool-1-thread-1" #12 prio=5 os_prio=0 cpu=142.12ms nid=0x1a4b waiting on condition`, `nid` is the native OS thread ID in hexadecimal. By mapping CPU-heavy PIDs from `top -H -p \<PID>` to hexadecimal, you identify the exact hotspot method trace. 3) Monitor Chain Resolution: Search for `waiting to lock \<0xADDR>` and grep for `locked \<0xADDR>` to construct lock dependency graphs. 4) JVM Deadlock Engine: HotSpot automatically scans for cyclic object monitor dependencies and appends a formal deadlock report to the dump tail.

## ⚡ Modern Java 25 LTS Kodu

## ⚡ Modern Java 25 Implementation

```java
// Java 25: Programatik Olarak Deadlock Tespiti ve Thread Dökümü Alma
import java.lang.management.ManagementFactory;
import java.lang.management.ThreadInfo;
import java.lang.management.ThreadMXBean;

public class ThreadDumpDiagnosticDemo {
    public static void scanForDeadlocks() {
        ThreadMXBean bean = ManagementFactory.getThreadMXBean();
        long[] deadlockedThreadIds = bean.findDeadlockedThreads();

        if (deadlockedThreadIds != null && deadlockedThreadIds.length > 0) {
            System.err.println("🚨 KRİTİK ALARM: " + deadlockedThreadIds.length + " Thread Deadlock'a
girdi!");
            ThreadInfo[] infos = bean.getThreadInfo(deadlockedThreadIds, true, true);
            for (ThreadInfo info : infos) {
                System.err.printf("Deadlock Thread: %s (ID: %d) beklediği kilit: %s, kilidi tutan:
%s%n",
                    info.getThreadName(), info.getThreadId(),
                    info.getLockInfo(), info.getLockOwnerName());
            }
        } else {
            System.out.println("✅ Deadlock tespit edilmedi. Sistem sağlıklı.");
        }
    }

    void main() {
        scanForDeadlocks();
    }
}
```

### ⚠️ Deadlock, Race Condition & Mülakat Tuzağı

#### 🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Production ortamında asla tek bir thread dump ile karar vermeyin! 10 saniye arayla en az 3 adet thread dump alın (ör. dump1, dump2, dump3). Eğer bir thread 3 dökümde de aynı satırda RUNNABLE durumundaysa sonsuz döngüdedir; eğer 3 dökümde de BLOCKED ise kilit açlığını gösterir.

### ⚠️ Concurrency Hazard & Interview Trap

#### 🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Never diagnose a production outage from a single thread dump! Always capture at least 3 consecutive snapshots spaced 5-10 seconds apart. A thread observed as `RUNNABLE` at the identical instruction line across all snapshots is trapped in an infinite loop; persistent `BLOCKED` states confirm lock starvation.

### 🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• top -H ile yüksek CPU kullanan thread ID'yi hex'e çevirin, jcmd Thread.print çıktısında 'nid' olarak aratıp sorunu nokta atışı tespit edin.

### 🎯 Key Takeaways for the Interview

• Convert high-CPU OS thread IDs to hex via `printf '%x'`, grep for `nid` in `jcmd Thread.print`, and trace the exact offending call stack.